

AI Tool(s) Used: Claude, ChatGPT, Gemini

1. MapGenerator.ts

我現在在座web Mario，可以請你幫我一個生成地圖的程式碼嗎？

Response:

```
const { ccclass, property } = cc._decorator;
```

```
@ccclass
```

```
export default class MapGenerator extends cc.Component {
```

```
    @property(cc.SpriteFrame)
```

```
    groundSpriteFrame: cc.SpriteFrame = null;
```

```
    @property(cc.SpriteFrame)
```

```
    brickSpriteFrame: cc.SpriteFrame = null;
```

```
    @property(cc.SpriteFrame)
```

```
    questionSpriteFrame: cc.SpriteFrame = null;
```

```
    @property(cc.Node)
```

```
    groundNode: cc.Node = null;
```

```
    @property(cc.Node)
```

```
    blocksNode: cc.Node = null;
```

```
tileSize = 32;
```

```
start() {
```

```
    this.generateGround();
```

```
    this.generateBlocks();
```

```
}
```

```
createTile(
```

```
    parent: cc.Node,
```

```
    spriteFrame: cc.SpriteFrame,
```

```
    x: number,
```

```
    y: number
```

```
){
```

```
    const node = new cc.Node();
```

```
    const sprite = node.addComponent(cc.Sprite);
```

```
    sprite.spriteFrame = spriteFrame;
```

```
node.parent = parent;

node.setPosition(x, y);

return node;
}
```

```
generateGround() {

    const width = 100;

    const groundY = -300;

    for (let i = 0; i < width; i++) {

        const x =

            i * this.tileSize -

            width * this.tileSize * 0.5;

        this.createTile(

            this.groundNode,
```

```
        this.groundSpriteFrame,  
        x,  
        groundY  
    );  
}  
}
```

```
generateBlocks() {
```

```
    const layout = [
```

```
        // x, y, type
```

```
        [200, -100, "brick"],
```

```
        [232, -100, "brick"],
```

```
        [264, -100, "question"],
```

```
        [296, -100, "brick"],
```

```
        [500, 0, "brick"],
```

```
        [532, 0, "question"],
```

```
        [564, 0, "brick"],
```

```
        [800, -50, "brick"],
```

```
        [832, -50, "brick"],
```

```
[864, -50, "brick"],
```

```
[1200, 50, "question"],
```

```
];
```

```
for (const item of layout) {
```

```
    const x = item[0];
```

```
    const y = item[1];
```

```
    const type = item[2];
```

```
    let spriteFrame = this.brickSpriteFrame;
```

```
    if (type === "question") {
```

```
        spriteFrame =
```

```
            this.questionSpriteFrame;
```

```
    }
```

```
    this.createTile(
```

```

        this.blocksNode,

        spriteFrame,

        x,

        y

    );

}

}

}

```

以上這個版本沒辦法順利生成地板與瓷磚，所以改用了更直觀的辦法，也就是用字串陣列來畫地圖，透過字串內容判斷現在生成的內容，後續敵人、static wall、不同種類 question block 的生成也是這樣達成的

Final version:

```

import GameData from "./GameData";

const { ccclass, property } = cc._decorator;

// =====

// 關卡地圖定義
// 每個字元代表一格 (32x32px) :
//   '0' = 空氣
//   '1' = 地板
//   '2' = 磚塊
//   '3' = 問號磚塊

```

[illegible]


```
@property(cc.Node) world: cc.Node = null;
```

```
//@property currentLevel: number = 0;
```

```
private currentLevel: number = 0;
```

```
readonly TILE_SIZE = 32;
```

```
onLoad() {
```

```
    cc.director.getPhysicsManager().enabled = true;
```

```
    this.currentLevel = GameData.currentLevel;
```

```
    this.loadLevel(this.currentLevel);
```

```
    cc.director.getPhysicsManager().gravity = cc.v2(0, -800);
```

```
}
```

```
loadLevel(levelIndex: number) {
```

```
    this.world.removeAllChildren();
```

```
    const map = LEVELS[levelIndex];
```

```
    if (!map) return;
```

```
    const rows = map.length;
```

```
    const cols = map[0].length;
```

```
// 根據地圖長度自動調整位置，讓地圖從左邊開始
```

```
this.world.setPosition(-480, -265);
```

```
for (let row = 0; row < rows; row++) {  
    for (let col = 0; col < cols; col++) {  
        const char = map[row][col];  
        const x = col * this.TILE_SIZE;  
        const y = (rows - 1 - row) * this.TILE_SIZE;  
        if (char === 'F') {  
            this.spawnPrefab(this.flagPrefab, x, y);  
        } else if (char === '3') {  
            this.spawnQuestionBlock(x, y, "coin");  
        } else if (char === '4') {  
            this.spawnPrefab(this.shortWallPrefab, x, y);  
        } else if (char === '5') {  
            this.spawnPrefab(this.tallWallPrefab, x, y);  
        } else if (char === '6') {  
            this.spawnEnemy(x, y);  
        } else if (char === '7') {  
            this.spawnQuestionBlock(x, y, "mushroom");  
        } else if (char !== '0') {  
            this.createTile(parseInt(char), x, y, char !== '1');  
        }  
    }  
}
```

```

        this.buildGroundColliders(map, rows, cols);
    }

    spawnQuestionBlock(x: number, y: number, type: string) {

        let block = cc.instantiate(this.questionBlockPrefab);

        block.parent = this.world;

        block.setPosition(
            x + this.TILE_SIZE / 2,
            y + this.TILE_SIZE / 2
        );

        let script = block.getComponent("QuestionBlock");

        if (script) {
            script.blockType = type;
        }
    }

    spawnPrefab(prefab: cc.Prefab, x: number, y: number) {
        if (!prefab) return;

        let node = cc.instantiate(prefab);
    }

```

```
node.parent = this.world;

node.setPosition(x + this.TILE_SIZE/2, y + this.TILE_SIZE/2);

//node.group = 'wall'; // 設定牆壁群組，讓敵人能識別
}
```

```
spawnEnemy(x, y) {
    if (!this.enemyPrefab) return;

    let enemy = cc.instantiate(this.enemyPrefab);

    enemy.parent = this.world;

    enemy.setPosition(x + this.TILE_SIZE / 2, y + 10);

    enemy.group = 'enemy'; // 設定敵人群組，讓玩家能識別
}
```

```
createTile(type: number, x: number, y: number, addCollider: boolean) {

    const node = new cc.Node();

    node.anchorX = 0.5; node.anchorY = 0.5;

    node.setPosition(x + this.TILE_SIZE/2, y + this.TILE_SIZE/2);

    const sprite = node.addComponent(cc.Sprite);

    sprite.sizeMode = cc.Sprite.SizeMode.CUSTOM;

    node.setContentSize(this.TILE_SIZE, this.TILE_SIZE);

    switch (type) {
```

```
        case 1: sprite.spriteFrame = this.groundSpriteFrame; break;

        case 2: sprite.spriteFrame = this.brickSpriteFrame; break;

    }

    if (addCollider) {

        const body = node.addComponent(cc.RigidBody);

        body.type = cc.RigidBodyType.Static;

        body.enabledContactListener = true;

        const collider = node.addComponent(cc.PhysicsBoxCollider);

        collider.size = cc.size(this.TILE_SIZE, this.TILE_SIZE);

    }

    node.parent = this.world;

}

// 切換到下一關 (供 GameManager 呼叫)

nextLevel() {

    const next = this.currentLevel + 1;

    if (next < LEVELS.length) {

        this.currentLevel = next;

        GameData.currentLevel = next;
```

```

        this.loadLevel(this.currentLevel);

    } else {

        cc.log('所有關卡完成 ! ');

    }

}

buildGroundColliders(map: string[], rows: number, cols: number) {

    for (let row = 0; row < rows; row++) {

        let startCol = -1;

        for (let col = 0; col <= cols; col++) {

            const isGround = col < cols && parseInt(map[row][col]) === 1;

            if (isGround && startCol === -1) {

                startCol = col;

            } else if (!isGround && startCol !== -1) {

                // 建立從 startCol 到 col-1 的合併碰撞盒

                const width = (col - startCol) * this.TILE_SIZE;

                const x = startCol * this.TILE_SIZE;

                const y = (rows - 1 - row) * this.TILE_SIZE;

                const node = new cc.Node('ground_collider');

                node.anchorX = 0;

```



```

        node.anchorY = 0;

        node.setPosition(x, y);

        node.setContentSize(width, this.TILE_SIZE);


        const body = node.addComponent(cc.RigidBody);

        body.type = cc.RigidBodyType.Static;

        body.enabledContactListener = true;


        const collider =
node.addComponent(cc.PhysicsBoxCollider);

        collider.size = cc.size(width, this.TILE_SIZE);

        collider.offset = cc.v2(width / 2, this.TILE_SIZE / 2);


        node.parent = this.world;

        startCol = -1;

    }

}

}

}

}

```

2. PlayerGenerator.ts

好那我現在可以開始加玩家進去了嗎？

Response:

當然可以！既然地圖已經完美就位，現在正是把我們的馬利歐（**Player**）放進遊戲世界的最佳時機。

要讓玩家順利動起來並能在地板上奔跑，我們需要完成兩個部分：在 **Cocos Creator** 編輯器中設定節點，以及撰寫玩家的控制腳本。

步驟 1：在編輯器中創建 **Player** 節點

請在 **Cocos Creator** 的層級管理器（**Hierarchy**）中進行以下設定：

創建節點：找到你原本就有的 **Player** 節點（如果沒有，在 **Canvas** 下右鍵創建一個空節點，命名為 **Player**）。

調整位置與大小：

將 **Player** 的 **Position** 設在畫面上方（例如 **X: 0, Y: 0**），讓他遊戲開始時空中掉下來。

設定 **Size** 為 **32 x 32**（與你的 **tileSize** 一致）。

添加圖片：給 **Player** 節點添加一個 **cc.Sprite** 組件，並把你的馬利歐圖片拖進 **Sprite Frame**。

添加物理組件（核心）：

點擊添加組件 -> **cc.RigidBody**（剛體）。將 **Type** 設為 **Dynamic**（動態），並勾選 **Fixed Angle**（固定角度，防止馬利歐跌倒時整個人旋轉）。

點擊添加組件 -> **cc.PhysicsBoxCollider**（物理碰撞盒）。將 **Size** 設為 **32 x 32**

步驟 2：編寫 **PlayerController** 腳本

在資源管理器（**Assets**）創建一個名為 **PlayerController.ts** 的 **TypeScript** 腳本，並貼上以下經典的橫捲軸移動代碼：

```
const { ccclass, property } = cc._decorator;
```

```
@ccclass
```

```
export default class PlayerController extends cc.Component {

    // 移動速度與跳躍力
    moveSpeed: number = 300;
    jumpForce: number = 600;

    // 控制狀態
    private leftDown: boolean = false;
    private rightDown: boolean = false;
    private isGrounded: boolean = false; // 是否踩在地上

    private rb: cc.RigidBody = null;

    onLoad() {
        this.rb = this.getComponent(cc.RigidBody);

        // 監聽鍵盤事件
        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN,
this.onKeyDown, this);
        cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP,
this.onKeyUp, this);
    }

    onDestroy() {
        // 取消監聽
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN,
this.onKeyDown, this);
        cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP,
this.onKeyUp, this);
    }

    onKeyDown(event: cc.Event.EventKeyboard) {
        switch (event.keyCode) {
```

```

        case cc.macro.KEY.a:
        case cc.macro.KEY.left:
            this.leftDown = true;
            break;
        case cc.macro.KEY.d:
        case cc.macro.KEY.right:
            this.rightDown = true;
            break;
        case cc.macro.KEY.w:
        case cc.macro.KEY.space:
            if (this.isGrounded) {
                // 給予向上的線性速度實現跳躍
                this.rb.linearVelocity = cc.v2(this.rb.linearVelocity.x,
this.jumpForce);

                this.isGrounded = false; // 跳起後離地
            }
            break;
    }
}

onKeyUp(event: cc.Event.EventKeyboard) {
    switch (event.keyCode) {
        case cc.macro.KEY.a:
        case cc.macro.KEY.left:
            this.leftDown = false;
            break;
        case cc.macro.KEY.d:
        case cc.macro.KEY.right:
            this.rightDown = false;
            break;
    }
}

```

```

update(dt: number) {
    // 獲取當前物理速度
    let velocity = this.rb.linearVelocity;

    // 根據按鍵計算左右速度
    if (this.leftDown) {
        velocity.x = -this.moveSpeed;
    } else if (this.rightDown) {
        velocity.x = this.moveSpeed;
    } else {
        velocity.x = 0; // 沒按鍵時橫向靜止
    }

    // 將計算後的速度還給剛體
    this.rb.linearVelocity = velocity;
}

// 物理引擎碰撞回調：當玩家碰到東西時觸發
onBeginContact(contact: cc.PhysicsContact, selfCollider:
cc.PhysicsCollider, otherCollider: cc.PhysicsCollider) {
    // 簡單判斷：如果碰撞發生的法線朝上，代表踩在東西表面
    let manifold = contact.getManifold();
    let normal = manifold.localNormal;

    // 注意：根據碰撞主體不同，法線方向可能有正負差異，這裡做
    個基礎判定
    if (normal.y < 0 || otherCollider.node.y < this.node.y) {
        this.isGrounded = true;
    }
}
}

```

final version:

```

import GameManager from "../GameManager";

const { ccclass, property } = cc._decorator;

@ccclass
export default class PlayerGenerator extends cc.Component {

    moveSpeed: number = 250;
    jumpForce: number = 650;

    @property(cc.Node)
    mainCamera: cc.Node = null;

    @property(cc.SpriteFrame)
    transformEffect: cc.SpriteFrame = null;

    @property(cc.AudioClip)
    jumpSound: cc.AudioClip = null;

    @property(cc.AudioClip)
    dieSound: cc.AudioClip = null;

    @property(cc.AudioClip)
    mushroomSound: cc.AudioClip = null;

    private isBig: boolean = false;
    private leftDown: boolean = false;
    private rightDown: boolean = false;
    private isGrounded: boolean = false;
    private groundContactCount: number = 0;
    private groundedCooldown: number = 0;

    private rb: cc.RigidBody = null;

```

```

private anim: cc.Animation = null;
private currentAnim: string = "";
private startPos: cc.Vec2 = cc.v2(0, 0);
private isInvincible: boolean = false;

onLoad() {
    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_DOWN,
this.onKeyDown, this);
    cc.systemEvent.on(cc.SystemEvent.EventType.KEY_UP,
this.onKeyUp, this);
    this.startPos = this.node.getPosition(); // 紀錄初始位置
}

start() {
    this.rb = this.getComponent(cc.RigidBody);
    this.anim = this.getComponent(cc.Animation);

    if (this.rb) {
        this.rb.bullet = true;
        this.rb.enabledContactListener = true;
    } else {
        cc.error("找不到 RigidBody ! ");
    }

    if (this.anim) {
        this.anim.play("player_idle");
    }
}

update(dt: number) {
    if (!this.rb) return;

    if (this.node.y < -600) {

```

```

        this.getHurt();
    }

    if (this.groundedCooldown > 0) {
        this.groundedCooldown -= dt;
        if (this.groundedCooldown <= 0) {
            if (this.groundContactCount <= 0) {
                this.isGrounded = false;
            }
        }
    }

    let velocity = this.rb.linearVelocity;

    if (this.leftDown) {
        velocity.x = -this.moveSpeed;
        this.node.scaleX = -1;
    } else if (this.rightDown) {
        velocity.x = this.moveSpeed;
        this.node.scaleX = 1;
    } else {
        velocity.x = 0;
    }

    this.rb.linearVelocity = velocity;
    this.updateAnimation(velocity);
}

becomeBig() {
    if (this.isBig) return;
    this.isBig = true;

    if (this.mushroomSound) {

```



```

        cc.audioEngine.playEffect(this.mushroomSound, false);
    }
    // 播特效
    this.playTransformEffect();

    cc.tween(this.node)
        .to(0.2, { scale: 1.5 })
        .start();

    // 15 秒後變小
    this.scheduleOnce(() => {
        if (this.isBig) {
            this.becomeSmall();
        }
    }, 15);
}

getHurt() {
    if (this.isInvincible) return;
    this.isInvincible = true;

    this.leftDown = false;
    this.rightDown = false;

    if (this.isBig) {
        // 變大狀態被打 → 先變小，不扣命
        this.becomeSmall();
        // 給一段無敵時間
        this.scheduleOnce(() => {
            this.isInvincible = false;
        }, 2.0);
        return;
    }
}

```

```
if (this.dieSound) {
    cc.audioEngine.playEffect(this.dieSound, false);
}

// 小狀態被打 → 扣命 + reborn
GameManager.instance.updateLife(-1);

let rb = this.getComponent(cc.RigidBody);
if (rb) rb.linearVelocity = cc.v2(0, 0);

cc.tween(this.node)
    .repeat(5,
        cc.tween().to(0.1, { opacity: 0 }).to(0.1, { opacity: 255 })
    )
    .call(() => {
        this.reborn();
    })
    .start();
}

becomeSmall() {
    this.isBig = false;
    // 播特效
    this.playTransformEffect();
    // 縮回原本大小
    cc.tween(this.node)
        .to(0.2, { scale: 1 })
        .start();
}

reborn() {
```

```
cc.Tween.stopAllByTarget(this.node);

let rb = this.getComponent(cc.RigidBody);

if (rb) {
    rb.linearVelocity = cc.v2(0, 0);
}

this.node.setPosition(this.startPos);

let mainCam = this.mainCamera || cc.find("Canvas/Main Camera");

if (mainCam) {

    let follow = mainCam.getComponent("CameraFollow");

    if (follow) {
        follow.resetCamera(this.node.x);
    }
}

this.node.opacity = 255;

this.isInvincible = false;
this.isGrounded = false;
this.groundContactCount = 0;
}

onDestroy() {
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_DOWN,
this.onKeyDown, this);
    cc.systemEvent.off(cc.SystemEvent.EventType.KEY_UP,
this.onKeyUp, this);
}
```

```

    }

    onKeyDown(event: cc.Event.EventKeyboard) {
        switch (event.keyCode) {
            case cc.macro.KEY.a:
            case cc.macro.KEY.left:
                this.leftDown = true;
                break;
            case cc.macro.KEY.d:
            case cc.macro.KEY.right:
                this.rightDown = true;
                break;
            case cc.macro.KEY.w:
            case cc.macro.KEY.up:
            case cc.macro.KEY.space:
                if (this.isGrounded) {
                    this.rb.linearVelocity = cc.v2(this.rb.linearVelocity.x,
this.jumpForce);

                    this.isGrounded = false;
                    this.groundContactCount = 0; // ← 確認有這行
                    this.groundedCooldown = 0; // ← 加這行
                    if (this.jumpSound) {
                        cc.audioEngine.playEffect(this.jumpSound, false);
                    }
                }
                break;
        }
    }
}

```

```

onKeyUp(event: cc.Event.EventKeyboard) {
    switch (event.keyCode) {
        case cc.macro.KEY.a:
        case cc.macro.KEY.left:

```

```

        this.leftDown = false;
        break;
    case cc.macro.KEY.d:
    case cc.macro.KEY.right:
        this.rightDown = false;
        break;
    }
}

```

```

updateAnimation(velocity: cc.Vec2) {
    let animState = "player_idle";

    if (!this.isGrounded) {
        animState = "player_jump";
    } else if (velocity.x !== 0) {
        animState = "player_walk";
    } else {
        animState = "player_idle";
    }

    if (this.currentAnim !== animState) {
        this.currentAnim = animState;
        if (this.anim) {
            this.anim.play(animState);
        }
    }
}

```

```

onBeginContact(contact, selfCollider, otherCollider) {
    let worldManifold = contact.getWorldManifold();
    let normal = worldManifold.normal;
    if (normal.y < -0.5) {
        this.groundContactCount++;
    }
}

```

```
        this.isGrounded = true;
        this.groundedCooldown = 0;
    }
}
```

```
onEndContact(contact, selfCollider, otherCollider) {
    let worldManifold = contact.getWorldManifold();
    let normal = worldManifold.normal;
    if (normal.y < -0.5) {
        this.groundContactCount--;
        this.groundedCooldown = 0.1;
    }
}
```

```
playTransformEffect() {
    if (!this.transformEffect) return;

    let effectNode = new cc.Node('effect');
    effectNode.parent = this.node.parent;
    effectNode.setPosition(this.node.x, this.node.y);

    let sprite = effectNode.addComponent(cc.Sprite);
    sprite.spriteFrame = this.transformEffect;
    sprite.sizeMode = cc.Sprite.SizeMode.CUSTOM;
    effectNode.setContentSize(80, 80); // ← 調這個數字控制大小

    // 閃爍後消失
    cc.tween(effectNode)
        .repeat(3,
            cc.tween().to(0.1, { opacity: 0 }).to(0.1, { opacity: 255 })
        )
        .call(() => effectNode.destroy())
        .start();
}
```

```
}  
}
```

這個版本主要是改善了玩家會穿過地板的問題，還有增加了玩家的動畫、被敵人攻擊、重生的邏輯

3. Firebase.ts

接下來我要做：獎金（最多 10%） Firebase（5%） 部署到 Firebase 頁面 會員機制 註冊、使用 Firebase 登入、儲存/恢復遊戲進度 排行榜（5%）

Response:

```
const { ccclass, property } = cc._decorator;
```

```
@ccclass
```

```
export default class FirebaseManager extends cc.Component {
```

```
    public static instance: FirebaseManager = null;
```

```
    // Firebase 物件（從 window 取得）
```

```
    private auth: any = null;
```

```
    private db: any = null;
```

```
    onLoad() {
```

```
        FirebaseManager.instance = this;
```

```
        // 初始化 Firebase（確保 SDK 已載入）
```

```
        let firebase = (window as any).firebase;
```

```
        if (!firebase) {
```

```
            cc.error("Firebase SDK 未載入！");
```

```
            return;
```

```
        }
```

```
        const firebaseConfig = {
```

```
            apiKey: "你的 apiKey",
```

```
            authDomain: "你的 authDomain",
```

```

        projectId: "你的 projectId",
        storageBucket: "你的 storageBucket",
        messagingSenderId: "你的 messagingSenderId",
        appId: "你的 appId"
    };

    if (!firebase.apps.length) {
        firebase.initializeApp(firebaseConfig);
    }

    this.auth = firebase.auth();
    this.db = firebase.firestore();

    cc.log("Firebase 初始化成功！");
}

// 註冊
signUp(email: string, password: string, callback: (success: boolean,
msg: string) => void) {
    this.auth.createUserWithEmailAndPassword(email, password)
        .then((userCredential) => {
            cc.log("註冊成功:", userCredential.user.email);
            callback(true, "註冊成功！");
        })
        .catch((error) => {
            cc.error("註冊失敗:", error.message);
            callback(false, error.message);
        });
}

// 登入
signIn(email: string, password: string, callback: (success: boolean, msg:
string) => void) {

```



```

    this.auth.signInWithEmailAndPassword(email, password)
      .then((userCredential) => {
        cc.log("登入成功:", userCredential.user.email);
        callback(true, "登入成功！");
      })
      .catch((error) => {
        cc.error("登入失敗:", error.message);
        callback(false, error.message);
      });
  }

  // 登出
  signOut() {
    this.auth.signOut().then(() => {
      cc.log("已登出");
    });
  }

  // 儲存遊戲進度
  saveProgress(score: number, unlockedLevel: number) {
    let user = this.auth.currentUser;
    if (!user) {
      cc.warn("未登入，無法儲存進度");
      return;
    }

    this.db.collection("players").doc(user.uid).set({
      email: user.email,
      score: score,
      unlockedLevel: unlockedLevel,
      updatedAt: (window as
any).firebase.firestore.FieldValue.serverTimestamp()
    }).then(() => {

```

```

        cc.log("進度儲存成功！");
    }).catch((err) => {
        cc.error("儲存失敗:", err);
    });
}

// 讀取遊戲進度
loadProgress(callback: (data: any) => void) {
    let user = this.auth.currentUser;
    if (!user) {
        cc.warn("未登入，無法讀取進度");
        callback(null);
        return;
    }

    this.db.collection("players").doc(user.uid).get()
        .then((doc) => {
            if (doc.exists) {
                cc.log("讀取進度成功:", doc.data());
                callback(doc.data());
            } else {
                callback(null);
            }
        })
        .catch((err) => {
            cc.error("讀取失敗:", err);
            callback(null);
        });
}

// 取得排行榜（前 10 名）
getLeaderboard(callback: (data: any[]) => void) {
    this.db.collection("players")

```

```

        .orderBy("score", "desc")
        .limit(10)
        .get()
        .then((querySnapshot) => {
            let results = [];
            querySnapshot.forEach((doc) => {
                results.push(doc.data());
            });
            callback(results);
        })
        .catch((err) => {
            cc.error("排行榜讀取失敗:", err);
            callback([]);
        });
    }

    // 取得目前登入的使用者
    getCurrentUser() {
        return this.auth ? this.auth.currentUser : null;
    }
}

```

final version:

```

const { ccclass, property } = cc._decorator;

@ccclass
export default class FirebaseManager extends cc.Component {

    public static instance: FirebaseManager = null;

    private auth: any = null;
    private db: any = null;

```

```

onLoad() {
    // 單例模式：確保全遊戲只有一個 FirebaseManager
    if (FirebaseManager.instance && FirebaseManager.instance !== this)
{
        cc.log("發現重複的 FirebaseManager，正在刪除...");
        this.node.destroy();
        return;
    }

```

```

        FirebaseManager.instance = this;
        cc.game.addPersistRootNode(this.node); // 跨場景不銷毀，這非常
重要

```

```

        this.initFirebase();
    }

```

```

initFirebase() {
    const fb = (window as any).firebase;
    if (!fb) {
        cc.error("Firebase SDK 未載入，請檢查 index.html");
        return;
    }

    const firebaseConfig = {
        apiKey: "AlzaSyC7OZ7nKcBt-o3pFrtdEbR6Izvyedbb9DQ",
        authDomain: "webmario-112034071.firebaseio.com",
        databaseURL: "https://webmario-112034071-default-
rtdb.firebaseio.com",
        projectId: "webmario-112034071",
        storageBucket: "webmario-112034071.firebaseio.com",
        messagingSenderId: "707683618961",
        appId: "1:707683618961:web:478316e6fba49e1ec8af30",

```

```

        measurementId: "G-D123Y3RCMG"
    };

    if (!fb.apps.length) {
        fb.initializeApp(firebaseConfig);
        cc.log("Firebase 初始化完成！");
    }

    this.auth = fb.auth();
    this.db = fb.database();
}

// 安全取得 db 的方法：如果 db 是 null 則嘗試修復
private getSafeDb() {
    if (this.db) return this.db;
    const fb = (window as any).firebase;
    if (fb && fb.database) {
        this.db = fb.database();
        return this.db;
    }
    return null;
}

saveProgress(score: number, unlockedLevel: number) {
    let user = this.auth ? this.auth.currentUser : null;
    if (!user) {
        cc.warn("未登入，無法儲存");
        return;
    }

    let db = this.getSafeDb();
    if (!db) {
        cc.error("無法取得 Database 物件");
    }
}

```

```

        return;
    }

    cc.log("正在存檔至 RTDB...", user.uid);
    db.ref('players/' + user.uid).set({
        email: user.email,
        score: score,
        unlockedLevel: unlockedLevel,
        updatedAt: (window as
any).firebase.database.ServerValue.TIMESTAMP
    }).then(() => {
        cc.log("RTDB 儲存成功！");
    }).catch((err) => {
        cc.error("RTDB 儲存失敗:", err);
    });
}

loadProgress(callback: (data: any) => void) {
    let user = this.auth ? this.auth.currentUser : null;
    let db = this.getSafeDb();

    if (!user || !db) {
        callback(null);
        return;
    }

    db.ref('players/' + user.uid).once('value')
        .then((snapshot) => {
            callback(snapshot.exists() ? snapshot.val() : null);
        })
        .catch((err) => {
            cc.error("讀取失敗:", err);
            callback(null);
        });
}

```

```
});  
}
```

```
getLeaderboard(callback: (data: any[]) => void) {
```

```
    let db = this.getSafeDb();
```

```
    if (!db) return callback([]);
```

```
    db.ref('players')
```

```
        .orderByChild('score')
```

```
        .limitToLast(10)
```

```
        .once('value')
```

```
        .then((snapshot) => {
```

```
            let results = [];
```

```
            snapshot.forEach((child) => {
```

```
                results.push(child.val());
```

```
            });
```

```
            callback(results.reverse());
```

```
        })
```

```
        .catch((err) => {
```

```
            cc.error("排行榜讀取失敗:", err);
```

```
            callback([]);
```

```
        });
```

```
    }
```

```
// 註冊
```

```
signUp(email: string, password: string, callback: (success: boolean, msg: string) => void) {
```

```
    this.auth.createUserWithEmailAndPassword(email, password)
```

```
        .then((userCredential) => {
```

```
            cc.log("註冊成功:", userCredential.user.email);
```

```
            callback(true, "註冊成功！");
```

```
        })
```

```
        .catch((error) => {
```

```

        cc.error("註冊失敗:", error.message);
        callback(false, error.message);
    });
}

// 登入
signIn(email: string, password: string, callback: (success: boolean, msg:
string) => void) {
    this.auth.signInWithEmailAndPassword(email, password)
        .then((userCredential) => {
            cc.log("登入成功:", userCredential.user.email);
            callback(true, "登入成功！");
        })
        .catch((error) => {
            cc.error("登入失敗:", error.message);
            callback(false, error.message);
        });
}

// 登出
signOut() {
    this.auth.signOut().then(() => {
        cc.log("已登出");
    });
}

/// 儲存遊戲進度
// saveProgress(score: number, unlockedLevel: number) {
//     let user = this.auth.currentUser;
//     if (!user) return;
//     cc.log("user:", this.auth.currentUser);
//     this.db.ref('players/' + user.uid).set({
//         email: user.email,

```



```

//          score: score,
//          unlockedLevel: unlockedLevel,
//          updatedAt: (window as
any).firebase.database.ServerValue.TIMESTAMP
//      }).then(() => {
//          cc.log("RTDB 儲存成功!");
//      }).catch((err) => {
//          cc.error("RTDB 儲存失敗:", err);
//      });
//  }

/// 讀取進度 (RTDB 版) - 修正原本會卡住的地方
// loadProgress(callback: (data: any) => void) {
//     let user = this.auth.currentUser;
//     if (!user || !this.db) {
//         cc.warn("未登入或資料庫未準備好");
//         callback(null);
//         return;
//     }

//     // 修改：使用 .ref().once('value') 取代 .collection().get()
//     this.db.ref('players/' + user.uid).once('value')
//         .then((snapshot) => {
//             if (snapshot.exists()) {
//                 cc.log("讀取進度成功:", snapshot.val());
//                 callback(snapshot.val());
//             } else {
//                 cc.log("無存檔資料");
//                 callback(null);
//             }
//         })
//         .catch((err) => {
//             cc.error("讀取失敗:", err);

```

```

//          callback(null);
//      });
//}

//// 取得排行榜 (RTDB 版)
// getLeaderboard(callback: (data: any[]) => void) {
//     if (!this.db) {
//         cc.error("Database 物件不存在");
//         return callback([]);
//     }

//     this.db.ref('players')
//         .orderByChild('score')
//         .limitToLast(10)
//         .once('value')
//         .then((snapshot) => {
//             let results = [];
//             snapshot.forEach((child) => {
//                 results.push(child.val());
//             });
//             // 轉為降序並回傳
//             callback(results.reverse());
//         })
//         .catch((err) => {
//             cc.error("排行榜讀取失敗:", err);
//             callback([]);
//         });
//}

// 取得目前登入的使用者
getCurrentUser() {
    return this.auth ? this.auth.currentUser : null;
}

```

}

最終版本主要是修改了跟 **leader board** 有關的邏輯，因為原本用的並不適用於 **firebase database**，而是 **firebase firestore**